DATA ANALYTICS

DATA MANAGEMENT, DATA VIZ & TEXT MINING

Les opérations Groupby

Opérations GroupBy

Groupby() :

La fonction groupby() est utilisée pour regrouper le DataFrame en fonction de catégories.

- Le DataFrame est divisé en groupes.
- Une fonction d'agrégation est appliquée à chaque colonne du DataFrame divisé.
- Les résultats sont combinés.

Considérez le DataFrame suivant :

	Gender	Score
0	female	85
1	male	88
2	female	95
3	male	80

Opérations GroupBy

Groupby() :

La colonne « Genre » contient deux valeurs, masculine et féminine.

- Divisons notre DataFrame en deux parties en fonction de la colonne « Genre » ;
- La première partie contiendra les lignes où Sexe = masculin
- La deuxième partie contiendra les lignes où Sexe = féminin

	Gender	Score
0	female	85
2	female	95

	Gender	Score
1	male	88
3	male	80

Opérations GroupBy

Groupby() :

- Si nous trouvons le score moyen des deux sexes, voici ce que nous obtenons :

Gender	Score
female	90

Gender	Score
male	84

- En combinant les deux résultats, nous obtenons le DataFrame suivant :

Gender	Score
female	90
male	84

Opérations GroupBy

Groupby() :

La fonction groupby() fonctionne exactement de la même manière, sauf qu'elle facilite les choses pour nous.

- Dans l'exemple donné, nous regroupons notre DataFrame sur la base de la colonne « Genre», et puis appliquons la fonction d'agrégation mean().

```
In [55]: df
Out[55]:
```

	Gender	Score
0	female	85
1	male	88
2	female	95
3	male	80

```
In [56]: df.groupby(x['Gender']).mean()
Out[56]:
```

Gender	Score
female	90
male	84

Opérations GroupBy

Groupby() :

Notez que les fonctions d'agrégation sont appliquées automatiquement sur toutes les colonnes du DataFrame sauf celle/s utilisée(s) pour regrouper le DataFrame.

```
df
```

	Gender	Math	English
0	female	85	80
1	male	88	88
2	female	95	92
3	male	80	95

```
df.groupby(['Gender']).mean()
```

Gender	Math	English
female	90.0	86.0
male	84.0	91.5

Opérations GroupBy

Groupby() :

A NOTER :

L'utilisation de groupby renvoie toujours un DataFrame dont l'index est constitué des colonnes utilisées pour regrouper les données. Pour éviter cela, il faut ajouter **'reset_index()'** à la fin de la ligne.

```
df.groupby(['Gender']).mean().reset_index()
```

	Gender	Math	English
0	female	90.0	86.0
1	male	84.0	91.5

Opérations GroupBy

Groupby() :

Les fonctions d'aggregation les plus utilisées sont :

- mean()
- sum()
- max()
- min()
- median()
- count()
- std() (standard deviation)

Notebook 6 : Opérations GroupBy

Exercice:

1. Lire les deux feuilles du fichier cereals.xlsx en Pandas.
2. Ajouter pour chacun des deux DataFrames une colonne ‘Type’ qui prend la valeur du nom de la feuille source du DataFrame.
3. Concaténer les deux DataFrames.
4. Retrouver pour chaque ‘Type’ le nom du produit le mieux noté.
5. Retrouver pour chaque ‘Type’ la valeur Moyenne des calories et proteins uniquement.

DATA MANAGEMENT AVEC PANDAS

Partie 2

Nettoyage des données

- Les données extraites du monde réel contiennent souvent des valeurs incorrectes, incomplètes, non pertinentes ou manquantes qui doivent être nettoyées.

- Le nettoyage peut être effectué en modifiant, remplaçant ou supprimant ces valeurs.

- Le nettoyage des données est un élément fondamental dans le cycle de vie de la science des données.

https://www.educative.io/blog/what-is-data-cleaning

Les valeurs manquantes

- Les données manquantes sont courantes dans les ensembles de données réels.
- Pour identifier les valeurs manquantes, nous utilisons isna() ou isnull()

Exemple :

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
0	Tom	Hanks	63.0	m	8.0	10.0
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Hugh	Jackman	51.0	m	NaN	NaN
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0

```python
# Identifier les valeurs manquantes
valeurs_manquantes = df.isna().sum()
print(valeurs_manquantes)
```

```
first_name          1
last_name           1
age                 1
sex                 1
pre_movie_score     2
post_movie_score    2
dtype: int64
```

Les valeurs manquantes

- Pour supprimer les lignes où se trouvent les valeurs manquantes, nous utilisons dropna()

Exemple :

```
df.dropna()
```

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
0	Tom	Hanks	63.0	m	8.0	10.0
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0

Les valeurs manquantes

- Pour remplir les cases des valeurs manquantes avec une valeur définie nous utilisons fillna(v)

Exemple :

```
5 df.fillna('Valeur')
```

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
0	Tom	Hanks	63.0	m	8.0	10.0
1	Valeur	Valeur	Valeur	Valeur	Valeur	Valeur
2	Hugh	Jackman	51.0	m	Valeur	Valeur
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0

```
1 df.dtypes

first_name           object
last_name            object
age                 float64
sex                  object
pre_movie_score     float64
post_movie_score    float64
dtype: object
```

→

```
5 df.fillna('Valeur').dtypes

first_name          object
last_name           object
age                 object
sex                 object
pre_movie_score     object
post_movie_score    object
dtype: object
```

Les valeurs manquantes

- Pour éviter le changement de type de la colonne, il faudra ajouter des valeurs par colonne concernée

Exemple :

```
df['age'].fillna(30)
```

```
0    63.0
1    30.0
2    51.0
3    66.0
4    31.0
Name: age, dtype: float64
```

Les valeurs manquantes

3 Stratégies pour traiter les valeurs manquantes:

1. Supprimer toutes les lignes ayant des valeurs manquantes –> A ne pas faire si pas beaucoup de données
   - Possibilité de se retrouver avec 0 observation
   - Les données ne seront pas évaluées

2. Supprimer toutes les variables ayant des valeurs manquantes –> A ne pas faire sauf si la colonne est pratiquement vide
   - Perte d'information
   - Il peut y avoir une raison expliquant pourquoi une variable possède des valeurs manquantes

3. Imputer les valeurs manquantes –> Meilleure stratégie composée de 3 techniques principales

Les valeurs manquantes

Techniques d'imputation pour variables quantitatives:

- Imputer par la valeur qu'il devrait normalement y avoir si on sait pourquoi une donnée est manquante
- Imputer par une valeur impossible mais dans la même échelle de grandeur que la variable
- Imputer par une valeur possible (ex: Moyenne | Médiane ou Moyenne | Médiane en fonction d'autres variables
  ATTENTION: Cela introduit un biais de sur-représentation de la valeur vis-à-vis de la population

Bonus : les imputations basées sur du machine learning –> Imputation prédite. Ex : régression, arbres de décision.

- Avantage : peut capturer des relations complexes entre variables.
- Inconvénient : souvent calculatoire, parfois opaque ("black box"), et le gain peut être marginal selon le contexte.

Conseil:

Créer une **variable indicatrice binaire** : 1 si la valeur a été imputée, 0 sinon.
Permet à l'algorithme d'utiliser l'information "manquante" comme un signal supplémentaire.

Les valeurs manquantes

Techniques d'imputation pour variables qualitatives:

1. Si vous savez pourquoi une donnée est manquante: Imputer par la catégorie qui y réfère
2. Imputer par la catégorie la plus fréquente en fonctions des variables (mode)
3. Renommer la catégorie en « Missing »

Traitement des NaN : Analyse classique vs Machine Learning

Aspect	Analyse descriptive / statistique	Machine Learning
Objectif principal	Comprendre, résumer, visualiser	Prédire / modéliser
Imputation	Moyenne, médiane, mode, logique métier	Moyenne/médiane, logique métier, ou ML-based
Valeur impossible	Rarement utilisée	Parfois utilisée pour marquer les NA
Variable indicatrice (NA flag)	Pas nécessaire	Recommandée pour signaler les NA
Impact du biais	Important à considérer	Important si la variable indicatrice n'est pas utilisée
But	Résultats interprétables et cohérents	Maximiser la performance prédictive

Notebook 7 : Valeurs manquantes

Les valeurs dupliquées

Les valeurs dupliquées peuvent fausser les analyses.

- Utilisez duplicated() pour identifier les lignes dupliquées.
- Détecter les doublons sur certaines colonnes seulement : df.duplicated(subset=['col1', 'col2'])
- Compter le nombre de lignes dupliquées duplicated().sum()

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
0	Tom	Hanks	63.0	m	8.0	10.0
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Hugh	Jackman	51.0	m	NaN	NaN
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0
0	Tom	Hanks	63.0	m	8.0	10.0

```
dataframe.duplicated()
```

```
0    False
1    False
2    False
3    False
4    False
5     True
dtype: bool
```

```
duplicated_mask = dataframe.duplicated()
dataframe[duplicated_mask]
```

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
5	Tom	Hanks	63.0	m	8.0	10.0

Les valeurs dupliquées

- Utilisez drop_duplicates() pour supprimer les lignes dupliquées.

Exemple :

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
0	Tom	Hanks	63.0	m	8.0	10.0
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Hugh	Jackman	51.0	m	NaN	NaN
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0
0	Tom	Hanks	63.0	m	8.0	10.0

```
dataframe.drop_duplicates()
```

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
0	Tom	Hanks	63.0	m	8.0	10.0
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Hugh	Jackman	51.0	m	NaN	NaN
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0

```
dataframe.drop_duplicates(keep='last')
```

	first_name	last_name	age	sex	pre_movie_score	post_movie_score
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Hugh	Jackman	51.0	m	NaN	NaN
3	Oprah	Winfrey	66.0	f	6.0	8.0
4	Emma	Stone	31.0	f	7.0	9.0
5	Tom	Hanks	63.0	m	8.0	10.0

Les valeurs aberrantes

Une anomalie dans les données, également appelée valeur aberrante (ou ‘outlier’), est une observation qui diffère sensiblement par rapport au modèle standard.

• Des anomalies dans les données peuvent survenir pour diverses raisons, erreur humaine, arrêt de système …

➢ Considérons la série donnée contenant la largeur d'un tableau mesurée par 5 élèves différents.

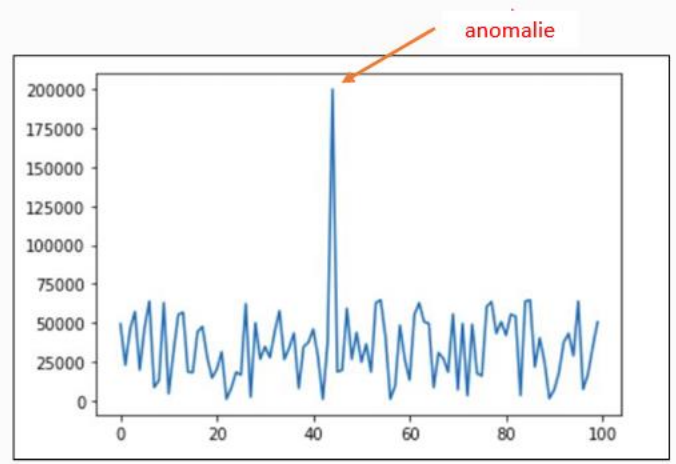
- La troisième valeur est significativement différente des autres –> outlier

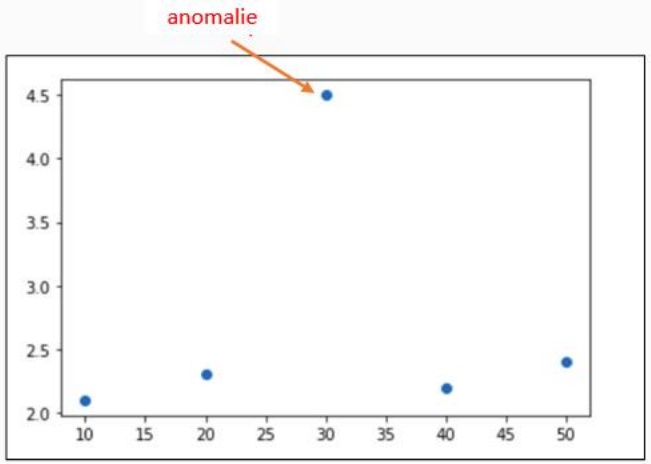
```
0    2.1
1    2.3
2    4.5
3    2.2
4    2.4
dtype: float64
```

Les valeurs aberrantes

•Pour détecter les outliers, il faut utiliser des méthodes statistiques comme la médiane ou la moyenne ou avec des visualisations

•Ensuite, il faut supprimer ou remplacer les valeurs aberrantes selon le cas suivant les même méthodes que pour les valeurs manquantes





Méthode utiles

Plusieurs méthodes utilisées dans des projets de Big Data & ML :

- **apply()** qui permet d'appliquer une fonction sur l'ensemble d'un dataframe ou une série
- **sort_values()** qui permet de trier un dataframe dans l'ordre croissant ou décroissant
- **corr()** qui permet d'afficher la matrice de corrélation entre les différentes variables
- **idxmin** & **idxmax** qui retourne l'index de la première occurrence du min / max dans la colonne choisie
- **value_counts** qui permet de compter le nombre d'occurrence de chaque valeur d'une colonne et les retourner dans l'ordre décroissant
- **replace** qui permet de remplacer une valeur par une autre valeur définie
- **unique** & **nunique** qui permet de retourner les valeurs uniques & le nombre des valeurs uniques
- **map** qui permet d'appliquer une fonction sur chaque élément d'une série
- **between** qui retourne True or False en fonction de si l'élement de la série se retrouve en les deux valeurs indiquées dans le between
- **sample** qui retourne un échantillon aléatoire d'éléments à partir des données du dataframe (utilisée pour décomposer notre dataframe en base de train & en base de test de façon aléatoire sans passer par la fonction train_test_split de scikit-learn).
- **nlargest** qui permet de sélectionner les n plus grandes valeurs

Notebook 8 : Méthodes utiles